

과제제안서 C211201 유영선		
과제명	LLM 기반의 동적 서사 생성을 활용한 로그라이크 게임	
팀명	직진만이 인생	
팀원	학번	이름
	C311183	황준호

1. 과제 개요

본 프로젝트는 Unity 기반 로그라이크 게임에 구조화된 생성형 AI를 결합하여, 플레이어의 선택과 이전 플레이 기록에 따라 다음 구간의 이야기와 사건 표현이 달라지는 동적 서사 시스템을 구현하는 것을 목표로 한다. 플레이 흐름은 메인 메뉴, 탐색 및 선택, 미니게임, 탄막 전투, 저장과 다음 회차의 이야기 변형으로 연결한다. 현재 구축한 JSON 기반 스토리와 환경 데이터, LLM API 연동, 저장 시스템, 전투 및 미니게임 구조를 통합하고, 학기 말에는 반복 플레이가 가능한 실행 빌드와 데이터 제작 절차를 완성한다.

2. 필요성 및 동기

AI 협업 개발 역량과 반복 가능한 제작 과정

생성형 AI를 실제 게임 제작 과정과 실행 시스템에 함께 적용하려면, 결과를 그대로 사용하는 방식보다 데이터 형식, 검증 기준, 저장 규칙을 먼저 설계해야 한다. 본 프로젝트에서는 기획, 구현, 테스트, 배포 준비까지의 전 과정을 반복하며 AI가 생성한 결과를 개발자가 검토하고 승인할 수 있는 제작 흐름을 만든다. 이를 통해 AI 도구를 활용하더라도 게임 규칙과 품질 기준은 개발자가 통제하는 개발 방식을 실습한다.

기존 게임과 서비스 분석

Reigns의 카드 선택 방식은 적은 입력으로도 선택 결과를 명확하게 전달하고, Undertale의 탄막 전투는 이야기 진행 중 별도의 조작 경험을 제공한다. AI Dungeon과 같은 생성형 서사 서비스는 텍스트 확장성이 높지만, 수작업으로 구성된 3D 환경, 구조화된 게임 상태, 미니게임 및 전투 규칙과의 결합은 별도로 설계해야 한다. 본 프로젝트는 이러한 장점을 참고하되, 자유 생성만으로 진행하지 않고 미리 정의한 JSON 구조와 게임 상태 안에서 이야기 변형이 일어나도록 범위를 제한한다.

- 차별점은 플레이어 기록을 다음 회차의 이야기 변형에 반영하면서도, terrain, 등장 인물, 선택지, 해금 조건, 전투 진입 조건을 구조화된 데이터로 유지하는 데 있다. 따라서 반복 플레이마다 표현은 달라질 수 있지만 핵심 진행과 게임 규칙은 검증 가능한 상태로 남는다.
- 또한 terrain별 환경과 사건을 서로 연결하고, 이야기 속 행동이 코인 베팅, 튜토리얼 상호작용, 탄막 회피

등 서로 다른 미니게임으로 이어지도록 구성한다. 이를 통해 생성 문장만 달라지는 게임이 아니라 환경, 선택, 규칙이 함께 변하는 플레이 경험을 구현한다.

반복 개발과 통합 검증

개발은 기능을 작은 단위로 분리하여 구현, 플레이 테스트, 수정 순서로 반복한다. 스토리 데이터, 환경 생성, 미니게임, 전투, 저장 기능을 독립적으로 확인한 뒤 통합하며, Git 브랜치와 커밋으로 변경 이력을 관리한다. Codex, CLI, MCP, 플러그인과 skill은 코드 생성 자체보다 프로젝트 탐색, Unity 조작, 문서화, 반복 테스트 절차를 표준화하는 데 활용한다.

3. 요소 기술

- 구현에 사용하는 공통 기술과 팀원별 중점 기술

공통 기술

1. Unity 6.3 LTS와 C#

- 역할: 게임 실행 환경, 씬 전환, 상태 관리, UI, 미니게임 및 전투 로직 구현
- 적용: 선택 중심 인터페이스, 3D terrain 탐색, 코인 기반 상호작용, 탄막 회피 전투를 하나의 플레이 흐름으로 통합
- 구조: 기능별 Controller와 데이터 클래스를 분리하여 기능 추가 시 기존 코드의 변경 범위를 줄임

2. Git과 GitHub Desktop

- 역할: 팀 단위 형상 관리, 기능별 작업 분리, 변경 이력과 통합 상태 관리
- 적용: 개인 기능을 브랜치와 커밋 단위로 관리하고, 씬 및 prefab 충돌을 확인한 뒤 통합

3. Codex CLI MCP 플러그인과 skill

- 역할: 코드베이스 분석, Unity Editor 자동화, 반복 테스트, 에셋 및 문서 제작 보조
- 적용: 작업별 지침과 도구를 분리하여 개발 환경을 표준화하고, 생성 결과는 실행 및 렌더링 결과로 검증

4. Blender Pixilart와 Unity 에셋

- 역할: 2D 및 3D 게임 에셋 제작과 편집
- 적용: terrain별 시각적 특징, 캐릭터, 상호작용 오브젝트, UI 자산 구성

유영선: 데이터 중심의 동적 서사 환경 및 미니게임 시스템

황준호: LLM 모델 연동과 탄막 전투 시스템

1. API 기반 LLM과 한국어 중심 모델

- 현재 외부 LLM API로 기존 이야기 카드의 일부 문장을 구조적으로 변경하고 결과를 JSON으로 저장하는 흐름을 사용한다. 이후 한국어 이야기 변경에 적합한 공개 모델을 크기별로 비교하고, 원문 이야기와 플레이 기록을 입력으로 받아 지정된 JSON 형식의 변형 이야기를 출력하도록 파인튜닝한다. 평가는 한국어 자연스러움, 맥락 일관성, 형식 준수율, 응답 시간과 실행 자원을 기준으로 진행한다.

- 기존 기술과 구현 자산의 재사용

- 유영선: 1학기에 구현한 선택형 스토리 UI, Story JSON 구조, LLM API 통신, 회차별 생성 콘텐츠 저장, 플레이 기록과 세이브 데이터, 코인 베틱 및 인벤토리 구조를 재사용한다. 이번 학기에는 terrain prefab과 JSON의 역할을 분리한 환경 시스템, Encounter별 Interaction과 Story 데이터, 튜토리얼 미니게임 구조를 추가하여 확장한다.
- 황준호: Undertale의 전투 전환과 탄막 회피 조작을 참고하고, 기존 BattleScene의 단계 실행, BattleBox, 패턴 데이터와 충돌 판정 구조를 재사용한다.

- 차별화 기술

- 유영선: terrain의 고정 지형은 prefab으로 제작하고, 등장 인물, 상호작용 위치, 이야기 연결, 선택지, 해금 조건은 JSON으로 분리한다. Excel은 전체 콘텐츠 목록과 관계를 검토하는 작성 원본으로 사용하고, 검증을 거친 JSON을 Unity가 읽도록 구성한다. LLM은 게임 전체를 자유 생성하지 않고 플레이한 카드와 허용된 필드만 변경하여 안정성과 재현성을 확보한다.
- 황준호: 기존 탄막 회피에 물체 베기, 패링, 화면 변화 등 새로운 조작 기반 패턴을 추가하고, 전투별로 다른 흐름과 입력 방식을 적용한다.

- 중점 기술

- 유영선: TerrainRepository와 TerrainBuilder를 중심으로 terrain별 환경과 Encounter를 등록하고, StoryRelay와 AI API가 플레이 기록을 다음 회차의 이야기 변형으로 전달하도록 구성한다. 앞으로 Excel과 JSON 사이의 항목 대응, 필수 참조 검증, 누락 탐지 절차를 만들고, 각 terrain의 시각 콘셉트와 미니게임, 이야기 사건을 하나의 콘텐츠 단위로 관리한다.
- 황준호: 상태 기반 전투 제어, 코루틴 기반 순차 실행, 동적 BattleBox 변형을 활용한다. B1, B2, B3의 흐름과 개별 탄막 패턴을 분리하여 마우스 전진, 물체 베기, 화면 파괴 연출 등 전투 방식을 추가하고 패턴 확장 시 기존 시스템의 변경 범위를 최소화한다.

4. 상세 내용

- 요구사항 시스템 설계 구현 시험평가의 네 단계로 공통 내용과 팀원별 담당 내용을 정리한다

4.1 요구 사항

유영선: terrain별 환경이 지정된 prefab으로 생성되고, NPC와 상호작용 오브젝트는 JSON 위치 및 참조에 따라 배치되어야 한다. 플레이어의 선택, 방문 사건, 획득 아이템, 해금 정보와 코인 상태는 저장 및 복원되어야 한다. 미니게임은 이야기 사건에서 진입하고 결과를 선택 및 보상 데이터에 반영해야 한다. LLM 응답은 요청한 카드만 변경하고 지정된 JSON 형식과 한국어 문맥을 지켜야 한다.

황준호: 전투 사건이 발생하면 지정된 전투 흐름과 패턴이 실행되어야 하며, 플레이어 입력, 충돌, 피해, 성공 및 실패 결과가 다음 씬과 저장 데이터에 정확히 전달되어야 한다.

공통 기능 및 품질 요구사항

- 기능: 선택형 이야기, terrain 탐색, Encounter, 미니게임, 전투, 저장과 다음 회차 이야기 변형이 끊김 없이 연결되어야 한다.
- 데이터: Excel 작성 항목과 JSON 필드의 대응 관계가 명확해야 하며, prefabId, placeId, 이야기 경로와 해금 키의 누락 및 중복을 검사해야 한다.
 - 반응성: LLM 통신 중 게임 진행 상태를 안전하게 유지하고, 실패나 시간 초과가 발생하면 기존 이야기로 계속 진행할 수 있어야 한다.
- 일관성: 생성된 문장은 인물, 장소, 이전 선택과 모순되지 않아야 하며 요청하지 않은 카드나 식별자를 변경해서는 안 된다.
- 확장성: 새로운 terrain, Encounter, 미니게임과 전투 패턴을 기존 코드의 대규모 수정 없이 데이터와 모듈 추가로 연결할 수 있어야 한다.
- 사용성: 선택 결과와 조작 피드백이 즉시 보이고, 전투와 미니게임의 난이도 및 진행 속도가 반복 플레이를 방해하지 않아야 한다.

4.2 시스템 설계

유영선: 시스템을 콘텐츠 데이터, 환경 구성, 플레이 진행, AI 이야기 변형, 저장 계층으로 나눈다. Excel 콘텐츠 맵은 기획 검토용 원본으로 사용하고 JSON은 Unity 실행 데이터로 사용한다. TerrainRepository가 JSON을 읽고 TerrainBuilder가 prefab과 배치 데이터를 조합하며, DecisionManager가 선택과 Encounter 흐름을 제어한다. StoryRelayManager는 플레이 기록과 변경 대상 카드를 묶어 LLM 요청을 만들고, 검증된 응답만 회차별 생성 콘텐츠로 저장한다. 황준호: 전투 시스템은 입력, 전투 제어, 패턴 실행, 물리 및 판정, 전투 데이터 계층으로 분리한다. 공통 BattleScene에서 전투 식별값에 맞는 Flow와 Pattern을 선택하고 완료 결과를 상위 게임 흐름으로 반환한다.

공통 실행 구조

- 표현 계층: 이야기 카드, terrain, 선택 UI, 미니게임, 전투 화면과 피드백 표시
- 입력 계층: 탐색, 선택, 코인 조작, 미니게임 및 현재 전투 단계에 맞는 사용자 입력 수집
- 진행 제어 계층: DecisionScene과 BattleScene의 상태, Encounter, 장과 에피소드 전환 및 생명주기 관리
- 콘텐츠 계층: terrain, Story, Interaction, 미니게임 설정과 전투 Pattern 데이터 제공
- 규칙 계층: 선택 조건, 해금, 보상, 충돌, 피해, 성공 및 실패 판정 처리
- 저장 및 AI 계층: 플레이 기록, 인벤토리, 코인, 해금 상태, 생성 이야기와 모델 처리 상태 저장

씬 및 콘텐츠 연결

- DecisionScene은 세션 데이터에 따라 현재 장과 에피소드의 terrain을 등록하고, JSON이 지정한 Encounter와 Story를 연결한다.
- BattleScene은 공통 씬을 사용하되 전투 식별값으로 B1, B2, B3의 흐름과 패턴을 선택하고 결과를 DecisionScene으로 반환한다.
- 각 단계는 고정 시간에만 의존하지 않고 대사 종료, 이동 완료, 미니게임 결과, 패턴 완료와 목표 도달 이벤트로 전환한다.

주요 자료구조

- Terrain과 Encounter 데이터: prefabId, placeId, 배치 위치, Story와 Interaction 경로, 요구 및 부여 해금 키
- 플레이 및 AI 데이터: 선택 기록, 플레이한 카드, 변경 대상, 회차 번호, 모델 처리 상태와 생성 콘텐츠 경로
- 전투 데이터: 전투 식별값, 단계 목록, 패턴, 지속 시간, 탄환 속도와 피해량, BattleBox, 이동 방식과 종료 조건

주요 알고리즘

- 콘텐츠 검증 및 로드: Excel에서 정리한 항목을 JSON과 비교하고, 참조 무결성을 확인한 뒤 terrain과 Encounter를 등록한다.
- 이야기 변형: 플레이 기록에서 대상 카드를 선택하고 LLM에 구조화된 요청을 보낸 뒤 카드 수, 식별자, 순번과 JSON 형식을 검증한다.
- 환경 스트리밍: 현재 진행 구간과 반경에 맞춰 terrain을 등록하고, 저장된 위치와 진행 상태를 복원한다.
- 미니게임 연결: Encounter의 행동과 결과에 따라 미니게임을 시작하고 보상, 해금 또는 다음 Story를 갱신한다.

- 전투 패턴 선택: 순차, 무작위, 가중치, 체력 및 이전 결과 조건을 이용해 다음 패턴을 결정한다.
- 예외 처리: LLM 실패, 누락 참조, 저장 데이터 오류가 발생하면 로그를 남기고 검증된 기본 데이터로 진행한다.

4.3 구현

1. 운영체제: Windows 10 및 11
2. 개발 언어: C# 및 데이터 정의용 JSON
3. 게임 엔진과 IDE: Unity 6.3 LTS, Visual Studio 2022
 - 데이터 도구: Excel 콘텐츠 맵, Unity Resources JSON, ScriptableObject와 prefab
 - AI와 자동화 도구: 외부 LLM API, 한국어 모델 파인튜닝 환경, Codex, CLI, MCP, 플러그인과 skill
 - 그래픽 도구: Blender, Pixilart 및 Unity 에셋 편집 도구
 - 협업 도구: Git, GitHub Desktop, Discord

4.4 시험평가

유영선: terrain별 prefab 로드와 JSON 배치 결과를 확인하고, Story 및 Interaction 경로, placeId, prefabId, 해금 키의 누락과 중복을 검사한다. 저장 후 재실행했을 때 진행 위치, 선택 기록, 인벤토리, 코인과 생성 이야기가 동일하게 복원되는지 확인한다. 미니게임 진입과 결과 반영은 정상 경로, 취소, 실패 조건으로 나누어 테스트한다.

황준호: B1, B2, B3의 단계 순서, 입력 방식, 탄막 충돌, 피해와 무적 시간, 성공 및 실패 판정, 씬 복귀 데이터를 반복 테스트한다.

- LLM 평가: 동일한 플레이 기록을 모델별로 입력하여 한국어 자연스러움, 앞뒤 맥락 일관성, JSON 형식 준수율, 요청 외 변경 비율을 비교한다.
- 성능 평가: API 및 로컬 모델의 평균 및 최대 응답 시간, 실패율, 메모리와 실행 자원을 기록하고 게임 진행을 막는 구간을 확인한다.
- 사용성 평가: 팀 내부와 외부 플레이 테스트에서 선택 이해도, terrain 탐색 동선, 미니게임과 전투의 난이도, 반복 플레이 의향을 확인하고 수정 이력을 남긴다.

5. 팀원별 역할

유영선: 선택형 스토리와 탐색 흐름, terrain prefab과 JSON 기반 환경 시스템, Encounter와 해금 데이터, 코인 및 튜토리얼 미니게임, 저장 및 회차별 이야기 변형 구조를 담당한다. Excel과 JSON을 이용한 콘텐츠 관리 절차를 구축하고, terrain별 환경과 이야기 및 미니게임을 추가한다. Codex, CLI, MCP, 플러그인과 skill을 활용해 개발 환경과 검증 절차를 정리하며, 한국어 중심 LLM의 크기별 성능 비교와 파인튜닝을 진행한다.

황준호: 외부 LLM 모델 연결 보조, BattleScene의 전투 흐름과 탄막 패턴 설계 및 구현, 전투별 조작 방식과 난이도 조정, 전투 결과의 씬 전환 및 저장 데이터 연동을 담당한다.

6. 주간별/팀원별 과제 수행 계획

설계요소 \ 주	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
유영선 요구사항 및 기존 구조 분석	D	D													
유영선 선택형 스토리와 저장 구조 정비		D	D	D											
유영선 LLM 이야기 변형 및 응답 검증			D	D	P	P	P								
유영선 terrain prefab과 JSON 환경 구성			D	D	P	P	P	P							
유영선 코인 및 튜토리얼 미니게임 구현				D	P	P	P	P							
유영선 Excel JSON 콘텐츠 관리 절차					P	P	P	P	P	P					
유영선 terrain별 환경 이야기 미니게임						P	P	P	P	P	P	P			
유영선 한국어 LLM 비교 및 파인튜닝								P	P	P	P	P	P		
유영선 통합 성능 사용성 테스트										P	P	P	P	P	
유영선 최종 통합 문서화 데모 준비													P	P	P
황준호 전투 흐름과 기본 패턴	D	D	D	D	P	P	P	P							
황준호 특수 조작 패턴과 난이도 조정					P	P	P	P	P	P	P	P	P		
유영선	D	D	D	D	P	P	P	P	P	P	P	P	P	P	P
황준호	D	D	D	D	P	P	P	P	P	P	P	P	P	P	P